AIM- To configure a GPIO pin of STM32F446RE as digital output and verify LED blinking operation using software delay routines.

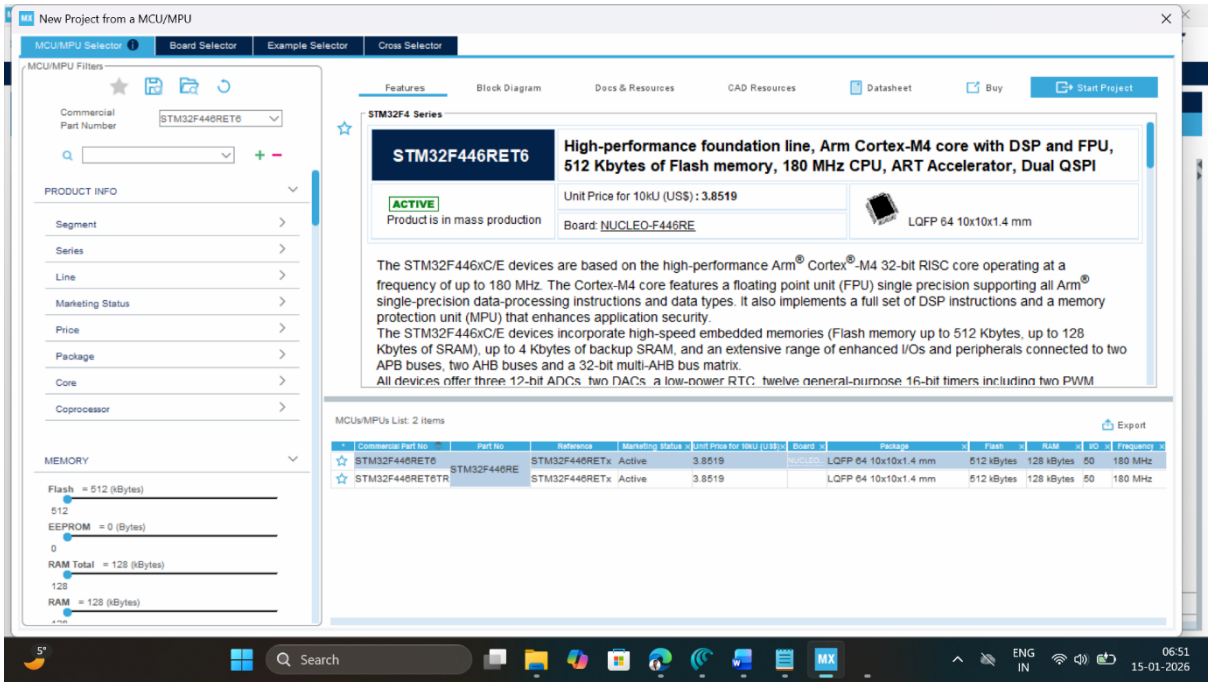
APPARATUS- STM32 Nucleo-F446RE development board, USB TypeA to MiniB cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

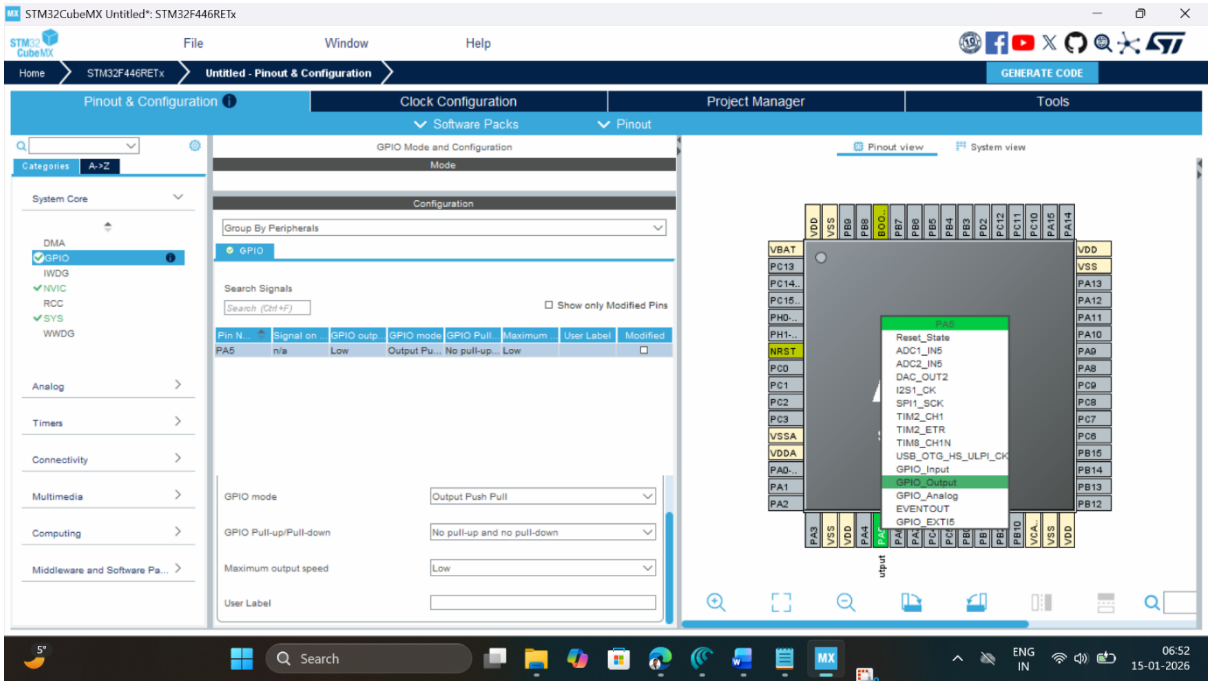
STM32F446RE provides multiple generalpurpose I/O (GPIO) ports (GPIOA–GPIOH) that can be configured as input, output, alternate function, or analog through GPIO control registers or HAL. On the NucleoF446RE board, the green user LED **LD2** is connected to Arduino D13, which corresponds to microcontroller pin **PA5** on most board revisions. When PA5 is configured as a **pushpull output**, the pin can be driven to logic high (near 3.3 V) or logic low (0 V), which in turn turns the LED on and off through an onboard currentlimiting. Using STM32CubeIDE and HAL, functions like HAL_GPIO_TogglePin() and HAL_Delay() can be used inside an infinite loop to generate a periodic LED blink without directly programming registers.

PROCEDURE

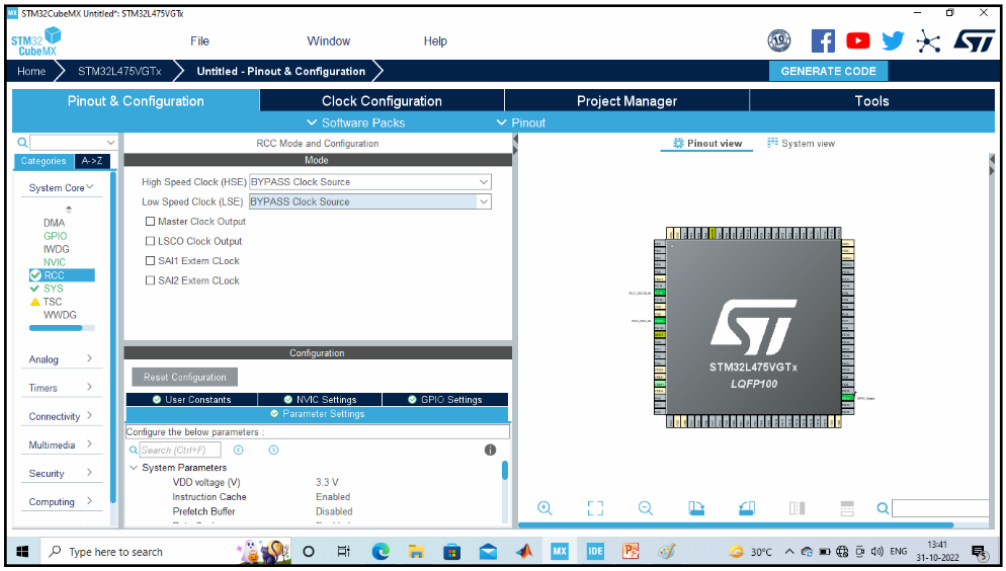
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



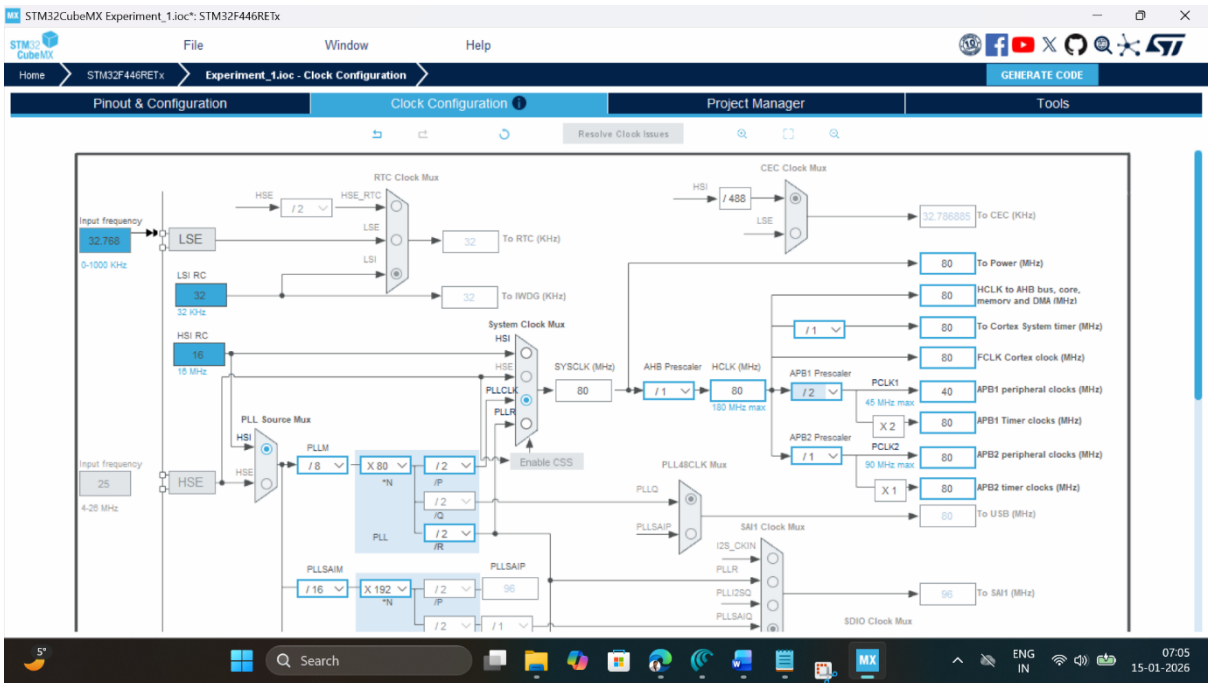
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED) relates to it in the board.



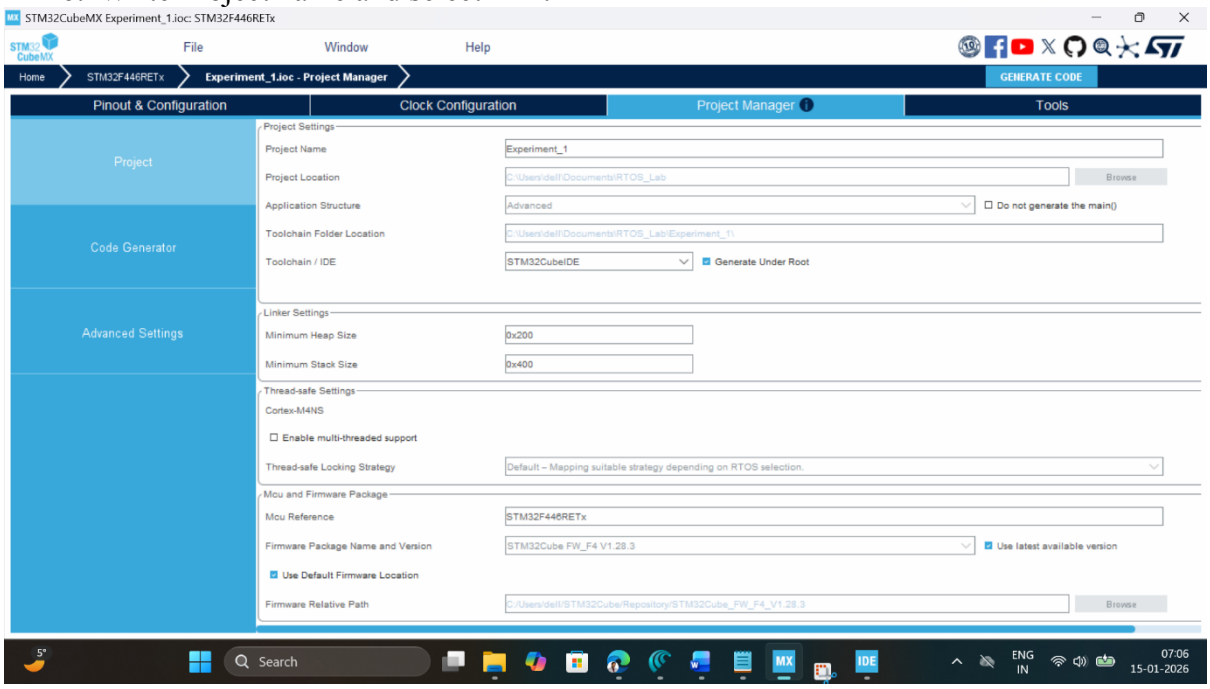
4. **Clock Selection**: Select the internal clock by clicking RCC and selecting BYPASS clock source



5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency.

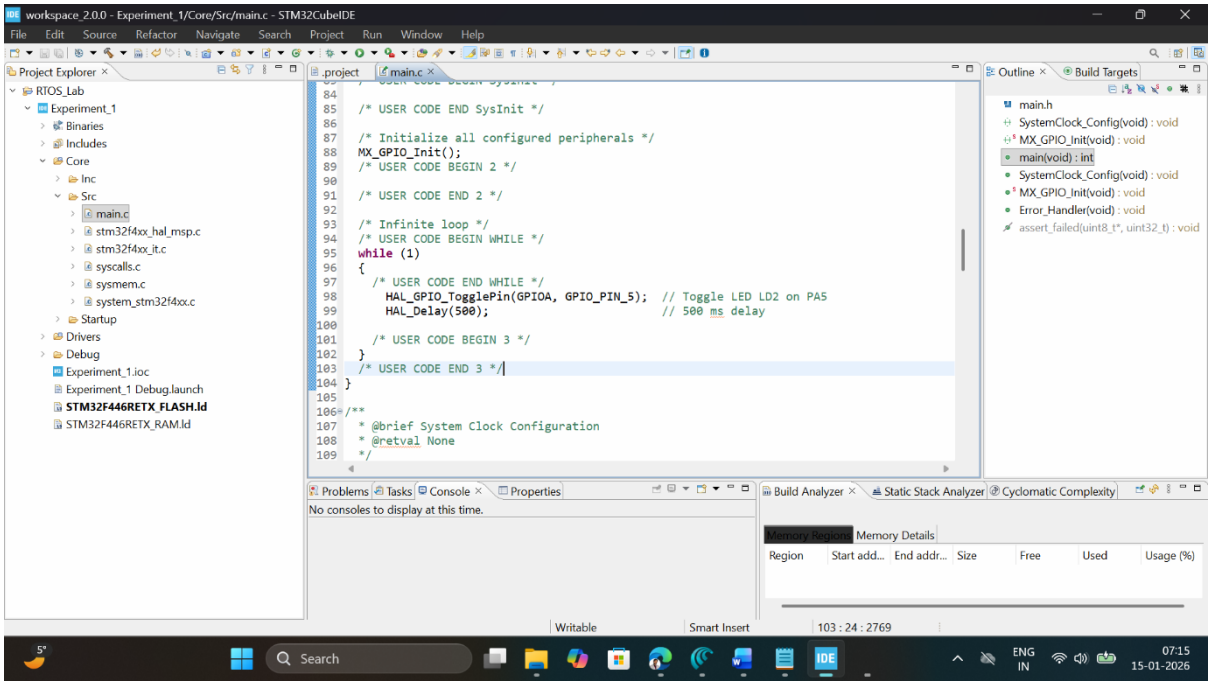


6. Write Project name and select IDE.



7. Click on generate code.
8. Click on open project.
9. Click ‘OK’ on the generated popup showing “successfully imported the project on the workspace” and the project will be successfully added to the cube IDE.
10. Open the main source code on Cube IDE by clicking on **main.c**
11. To generate bin file and hex file, right click on project and click properties.
12. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click “Apply and Close ".
13. After configuration write only below two instructions in the user while loop and compile the program and ensure there will be zero error after compiling.

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_Delay(500);
```



14. To blink the connected LED on the board, connect the board, click on build, click ‘ok’ and click ‘switch’ on popups.
15. Now you are in debugging mode, click ‘Resume’ to execute the program.

OBSERVATION: Change the delay value (e.g., 200 ms, 1000 ms), rebuild, rerun, and note the change in blink speed and fill the observation table.

S. No.	GPIO pin used	Mode / Type / Pull	Delay value (ms)	Approx. blink period (s)	Comment (slow/fast/visible)
1	PA5	Output, PP, No PU/PD	100	0.1s + 0.1s = 0.2 s	fast
1	PA5	Output, PP, No PU/PD	200	0.2 s + 0.2 s = 0.4 s	visible
2	PA5	Output, PP, No PU/PD	500	1 s	visible
3	PA5	Output, PP, No PU/PD	1000	1.5 s	slow

RESULT

During this experiment, the PA5 pin of the STM32F446RE microcontroller on the Nucleo-F446RE board was configured as a digital output using STM32CubeIDE. The built-in LED LD2

was successfully toggled, resulting in a blinking pattern created with software delays through HAL functions. This verified the correct setup and operation of the GPIO pin.

The use of HAL library routines such as HAL_GPIO_TogglePin() and HAL_Delay() played a crucial role in simplifying GPIO control. These functions provide a higher level of abstraction, eliminating the need for direct manipulation of hardware registers and allowing the developer to concentrate on program logic.

The experiment further highlighted important GPIO configuration parameters, especially the selection of output driver mode. The push-pull configuration, used in this case, enables the pin to drive both high and low voltage levels, making it suitable for LED control. While pull-up and pull-down resistors were unnecessary for this output configuration, their importance in stabilizing input signals was clearly understood.

Another concept reinforced during the practical was the relevance of GPIO speed and clock enabling. Although speed settings do not significantly affect LED blinking, they become critical in applications involving high-speed data transfer. The integration of CubeMX streamlined this process by automatically generating peripheral initialization code.

Overall, this experiment strengthened my understanding of GPIO operation in STM32 microcontrollers and demonstrated how proper configuration directly influences hardware behavior.